28. a. *Singly Linked List – TraverseListReverseOrderRecursive – Program Analysis and Time Complexity* .

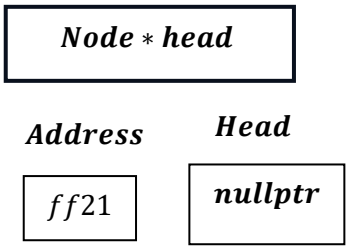
Program: –

```
void TraverseListReverseOrderRecursive(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {

        if ((*head)->next != nullptr)
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
….
 case 26:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Recursive" << endl;
            TraverseListReverseOrderRecursive(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

1) *Linked List is Empty*:

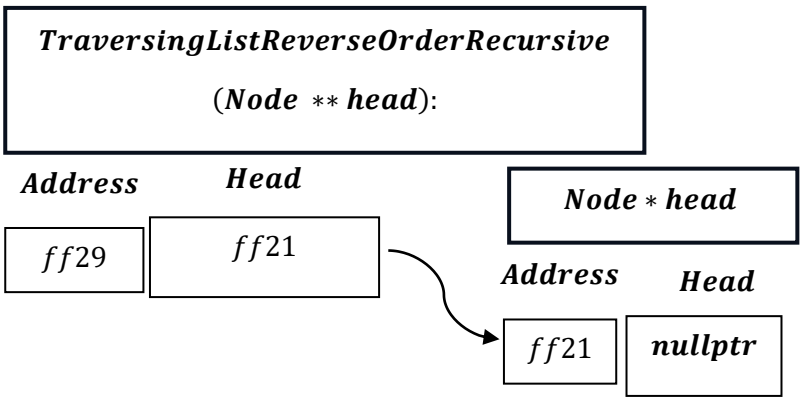
Pictorially depicted as:




```
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
```

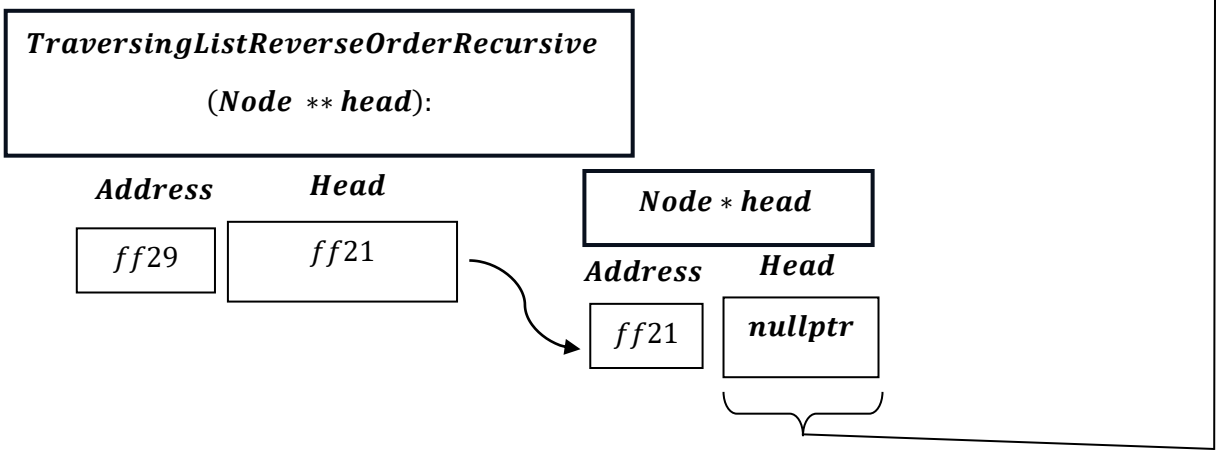
TraversingListReverseOrderRecursive(&***head***: ***ff21***);

```
void TraverseListReverseOrderRecursive (Node **head) {….}
```



$if(*head == nullptr)$ *if it is true*

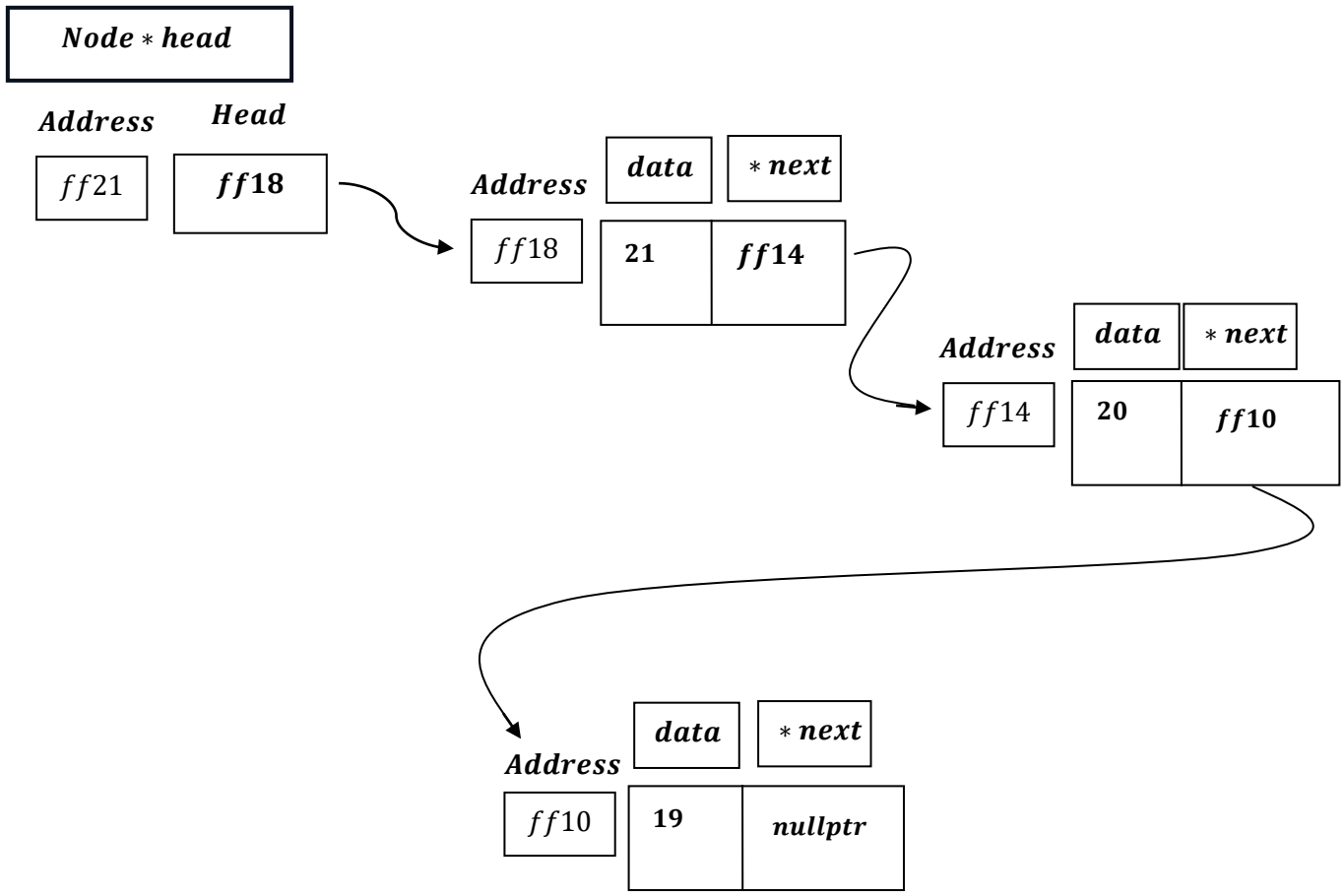
If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the `TraversingListReverseOrderRecursive` function.



If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

---

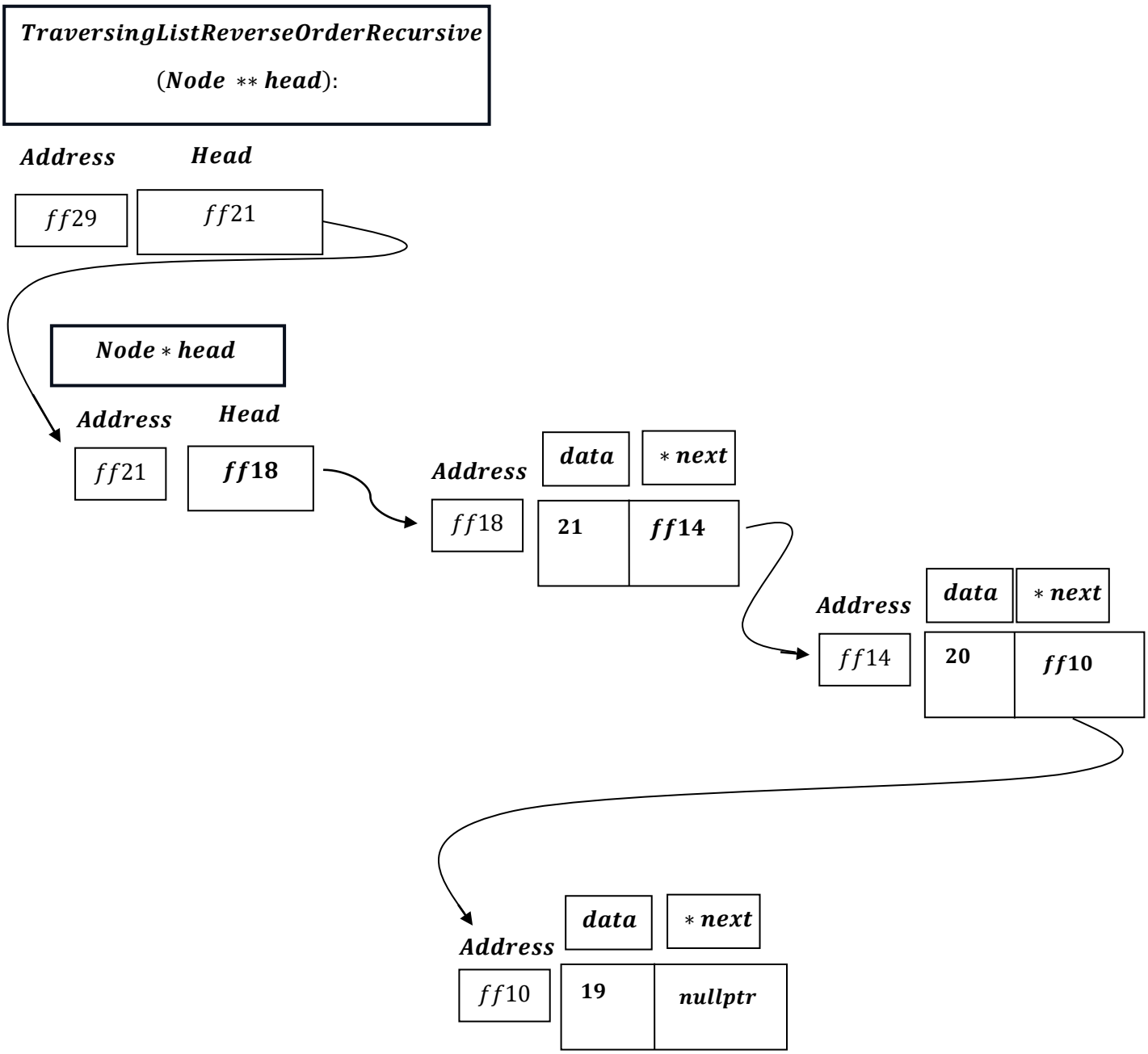
2)*Linked List is not Empty*:



Function Call:

```
TraverseListReverseOrderRecursive(&head);
```

$i.e. TraversingListReverseOrderRecursive(\&head: ff21)$



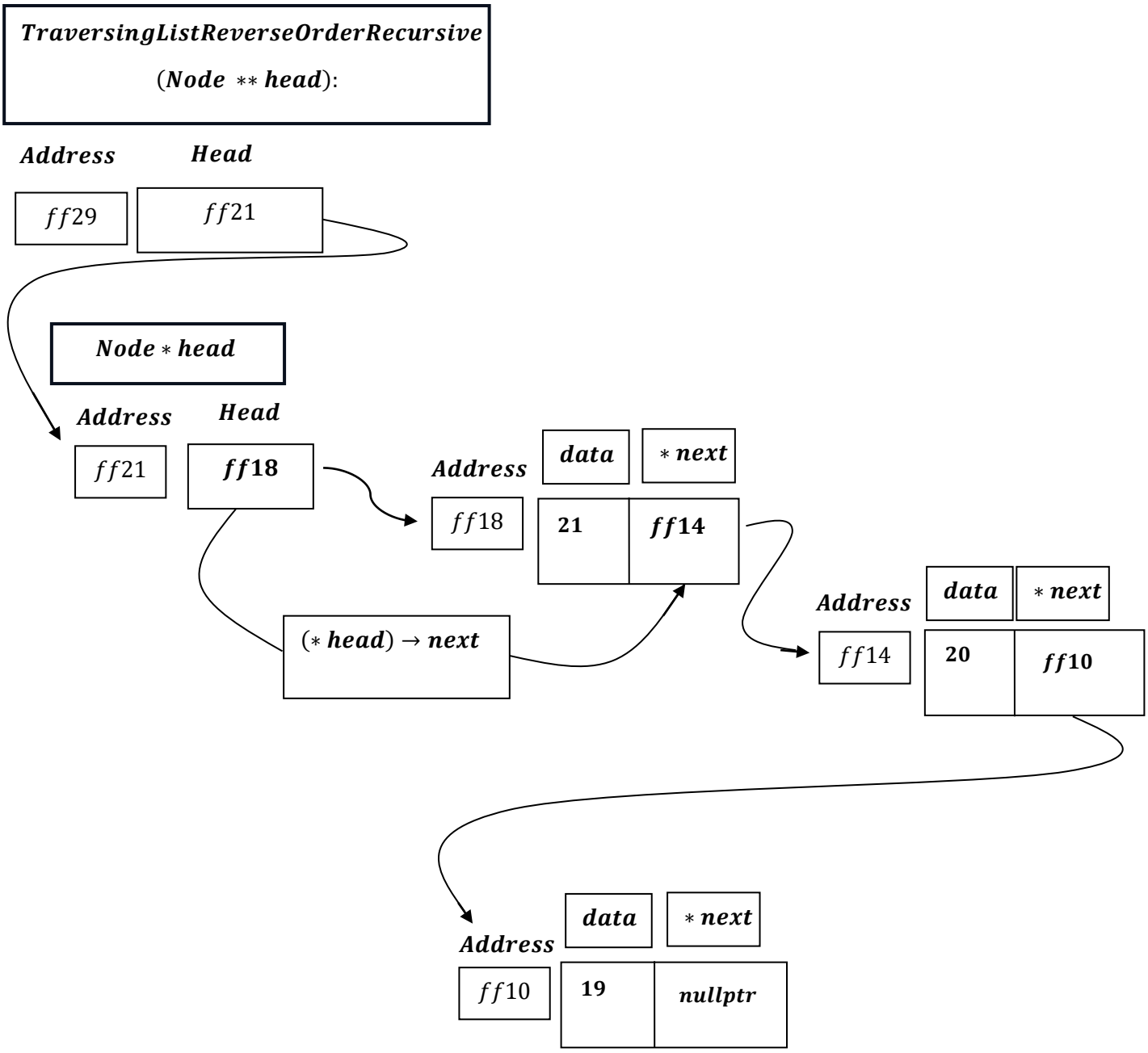

```
void TraverseListReverseOrderRecursive(Node **head) {
    if (*head == nullptr){ ….}

    else
    {
```

```
if ((*head)->next != nullptr)
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
```

```
if ((*head)->next != nullptr) {….}
```

i.e.,



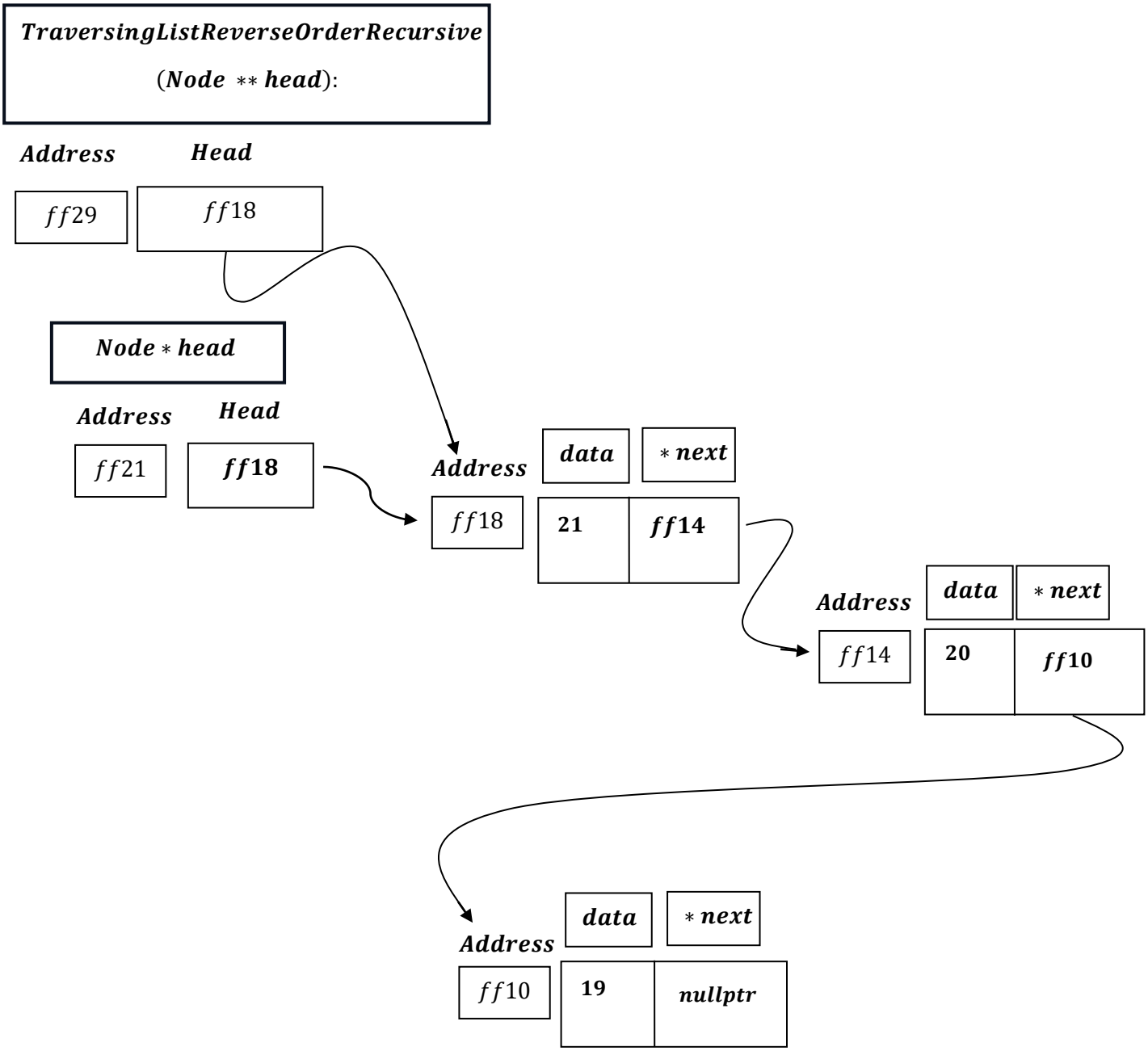
$Node * head$ *has value i.e. Address* $\rightarrow$ $ff18$ *and* $ff18 \rightarrow next$ *is* $ff14$ *i.e. not nullptr, then*:

Recursive Function call:

```
TraverseListReverseOrderRecursive(&((*head)->next));
```

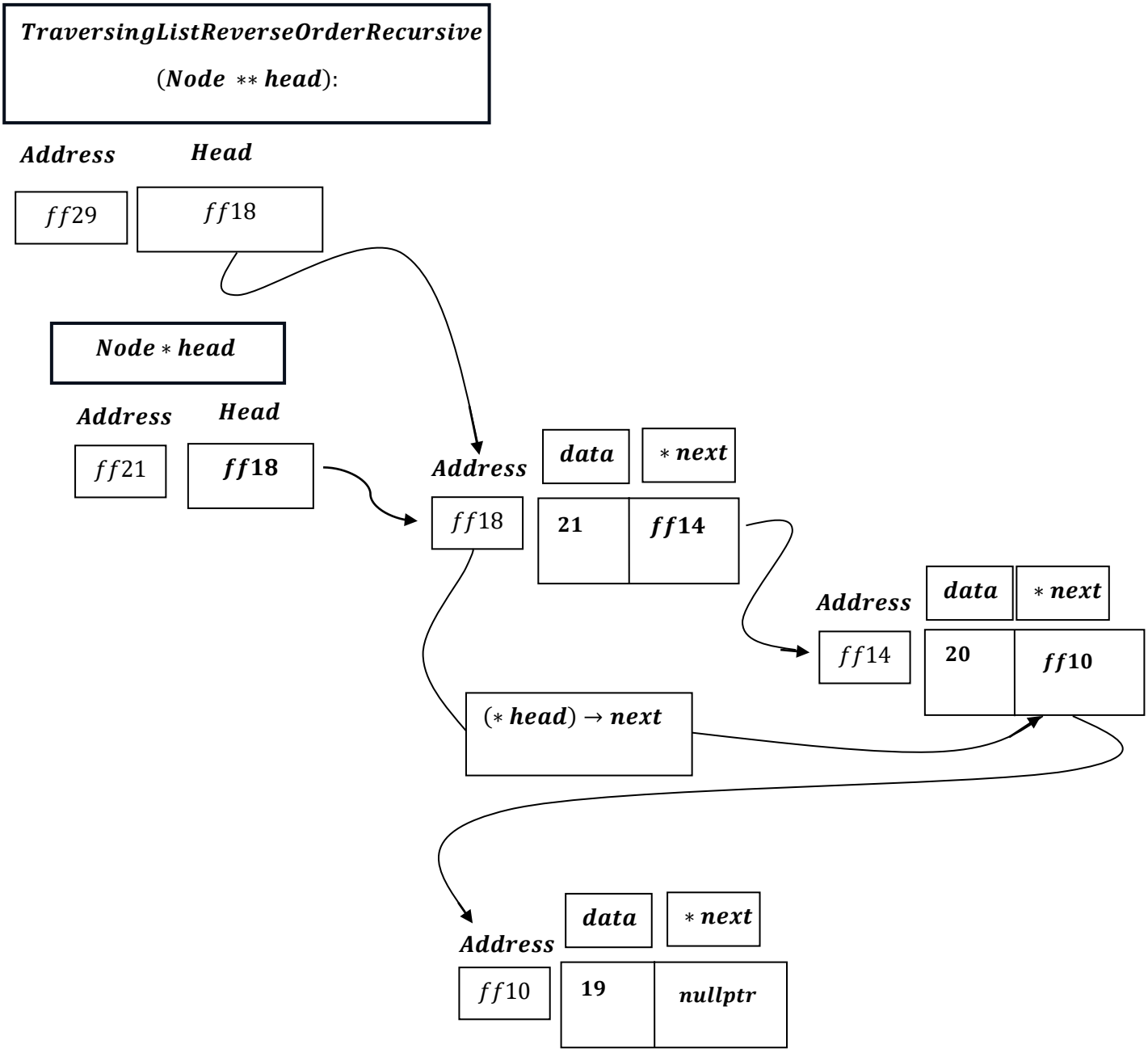
$ff50$	$Node ** head = \&((* head) \rightarrow next) = ff18$
$ff46$	$Main\ Func: Node ** head = \&head = ff21$

i.e. in Stack it will store the address where $(* head) \rightarrow next; \rightarrow ff14$ *is stored and* $Node ** head = \&((* head) \rightarrow next) = \&(ff14) = ff18$.



```
if ((*head)->next != nullptr) {….}
```

i.e.,



*Note, now, Node ** Head doesnot point to Node * Head , therefore , now , (* head) will dereference from address ff18 done by dereference operator `` * `` and Node * will point to ff14 and (* head) → next; reflects ff14 → next i.e., ff10 which is not nullptr , hence again recursive call takes place.*

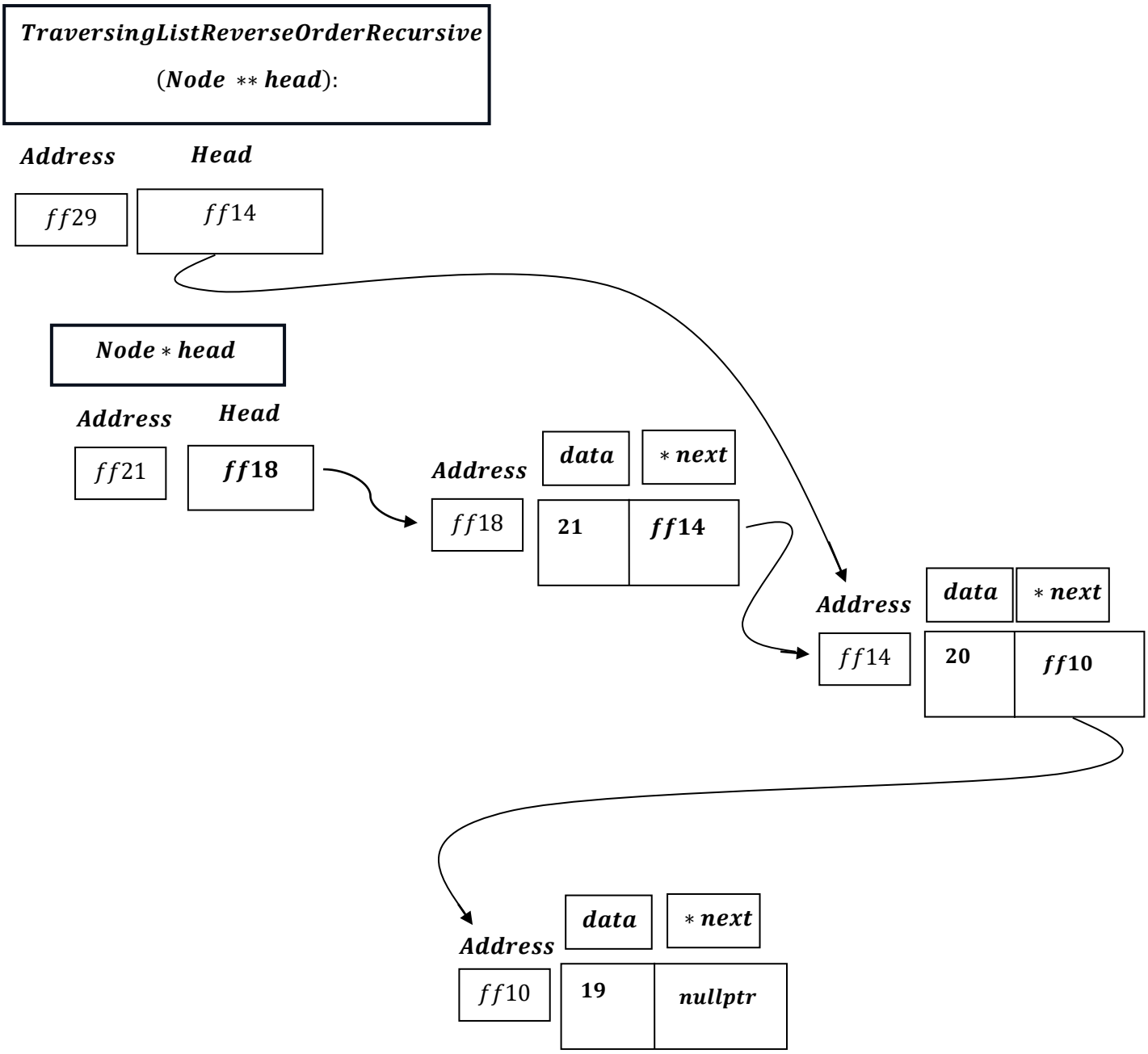
Recursive Function call:

```
TraverseListReverseOrderRecursive(&((*head)->next));
```

$ff54$	$Node ** head = \&((*head) \rightarrow next) = ff14$
$ff50$	$Node ** head = \&((*head) \rightarrow next) = ff18$
$ff46$	$Main\ Func: Node ** head = \&head = ff21$

$i.e.$ ***in Stack it will store the address where*** $(*head) \rightarrow next; \rightarrow ff10$ ***is stored and***

$Node ** head = \&((*head) \rightarrow next) = \&(ff10) = ff14.$

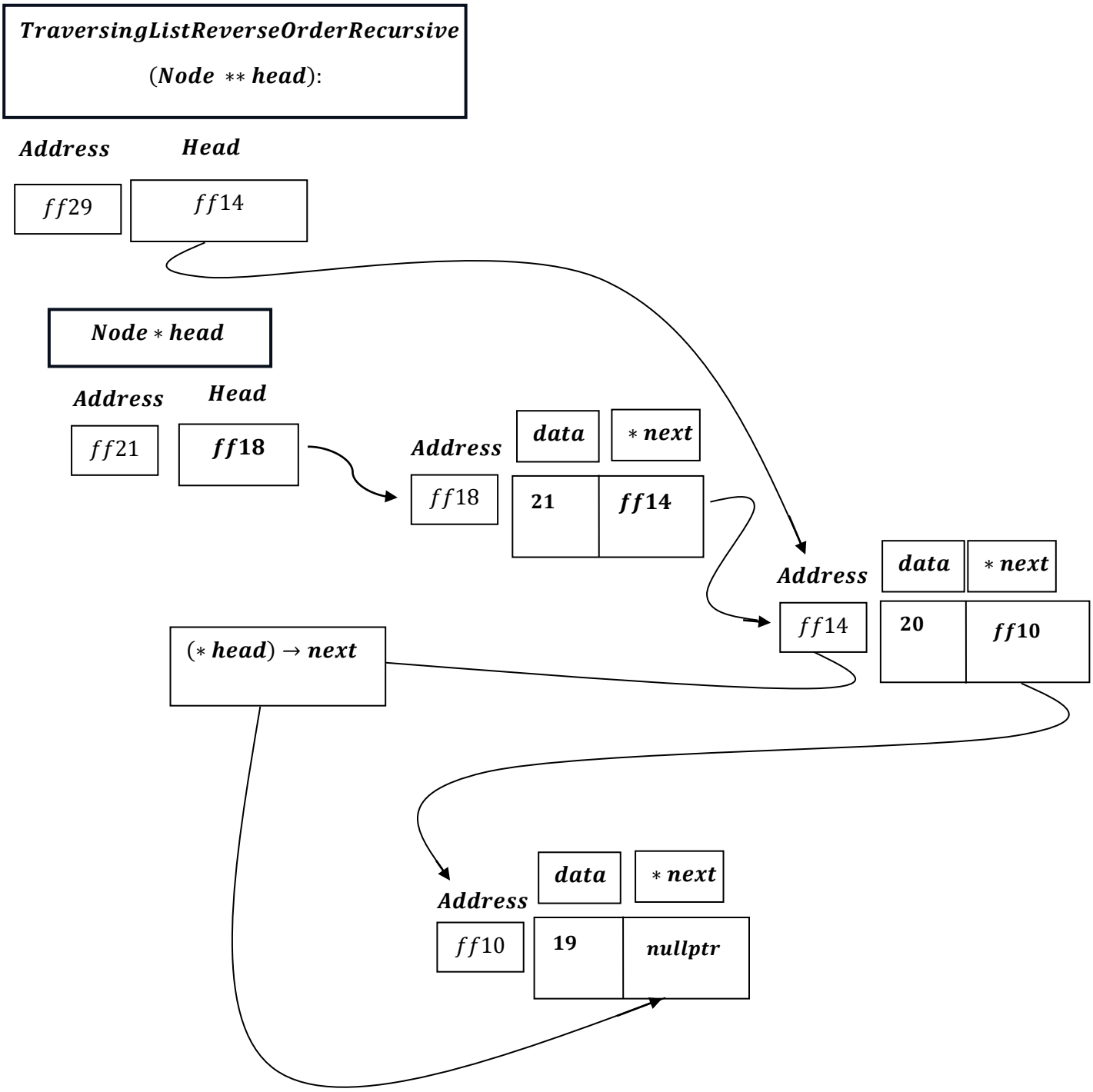


$Note, now, Node ** Head\ doesnot\ point\ to\ Node * Head\ , therefore\ , now\ ,$

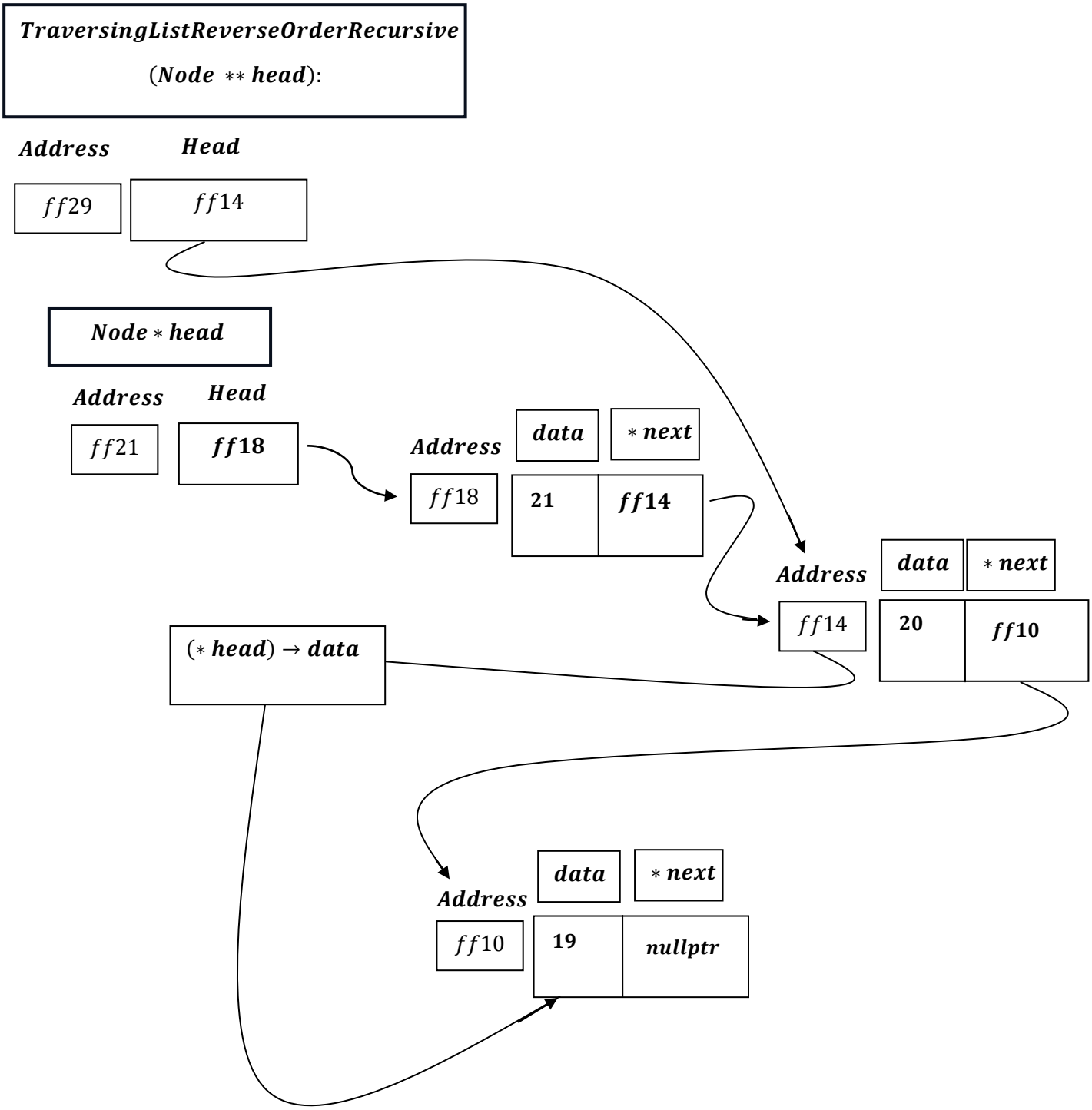
$(* head)\ will\ dereference\ from\ address\ ff14\ done\ by\ dereference\ operator$

$``*``\ and\ Node * will\ point\ to\ ff10\ and\ (* head) \rightarrow next; reflects\ ff14 \rightarrow next$

$i.e., ``nullptr``\ which\ is\ \ nullptr\ , hence\ again\ recursive\ will\ stop\ here\ and\ exit.$

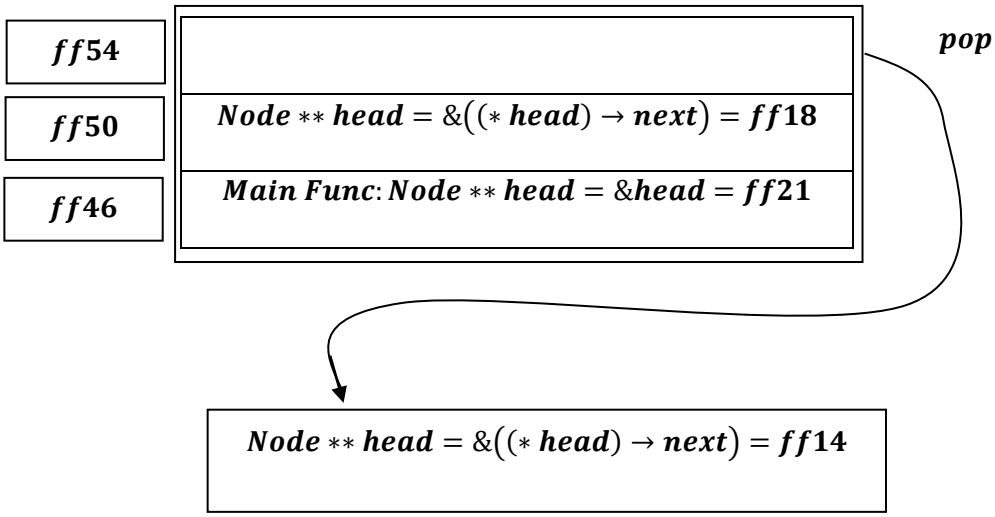


$Now, we\ will\ print\ (*\ head) \rightarrow data;$



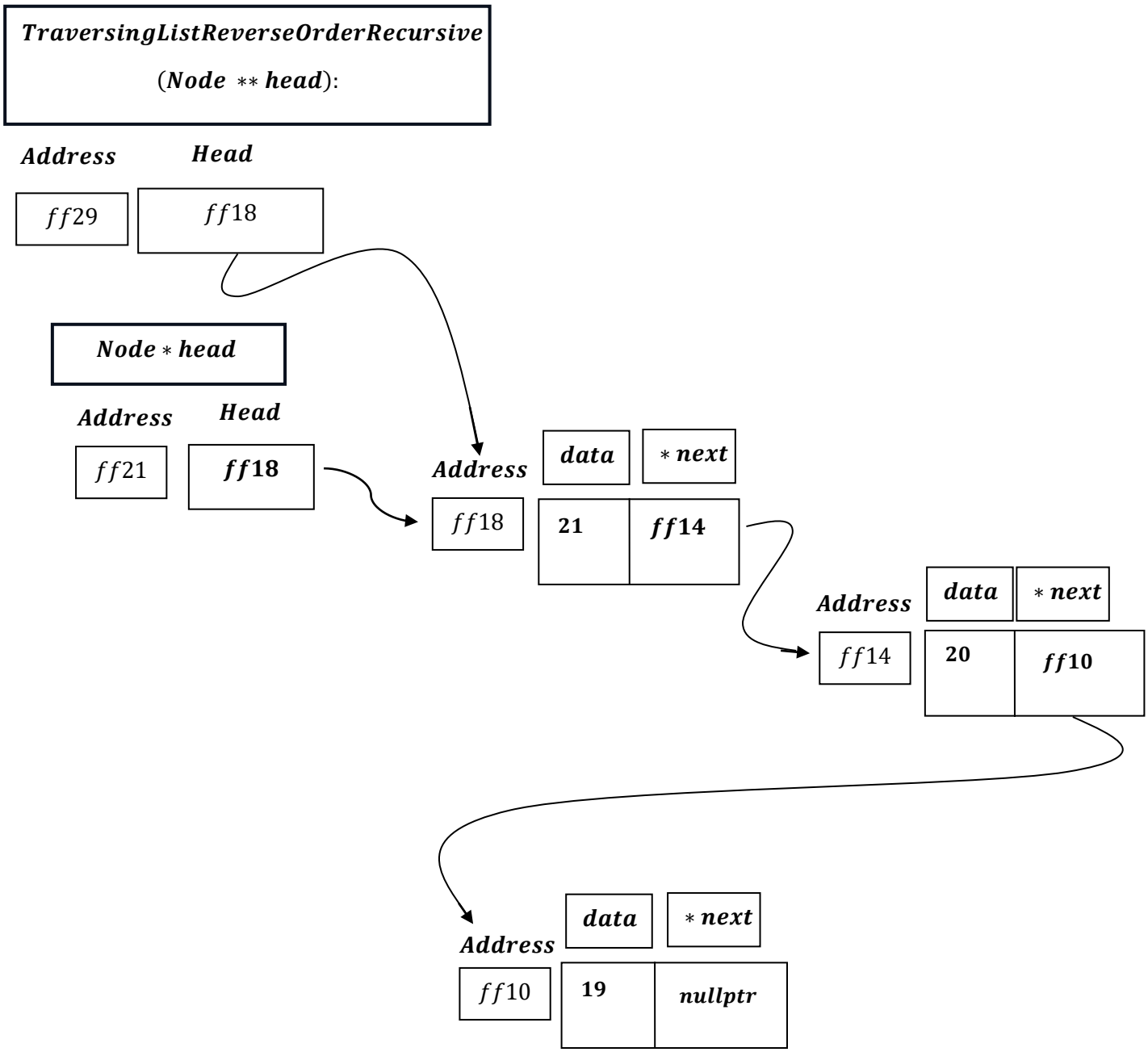
$Hence\ (*\ head) \rightarrow data;\ will\ print\ 19.$

$\boldsymbol{Next, Node ** head = \&((* head) \rightarrow next) = ff14, will\ pop\ out\ from\ the\ stack}$.

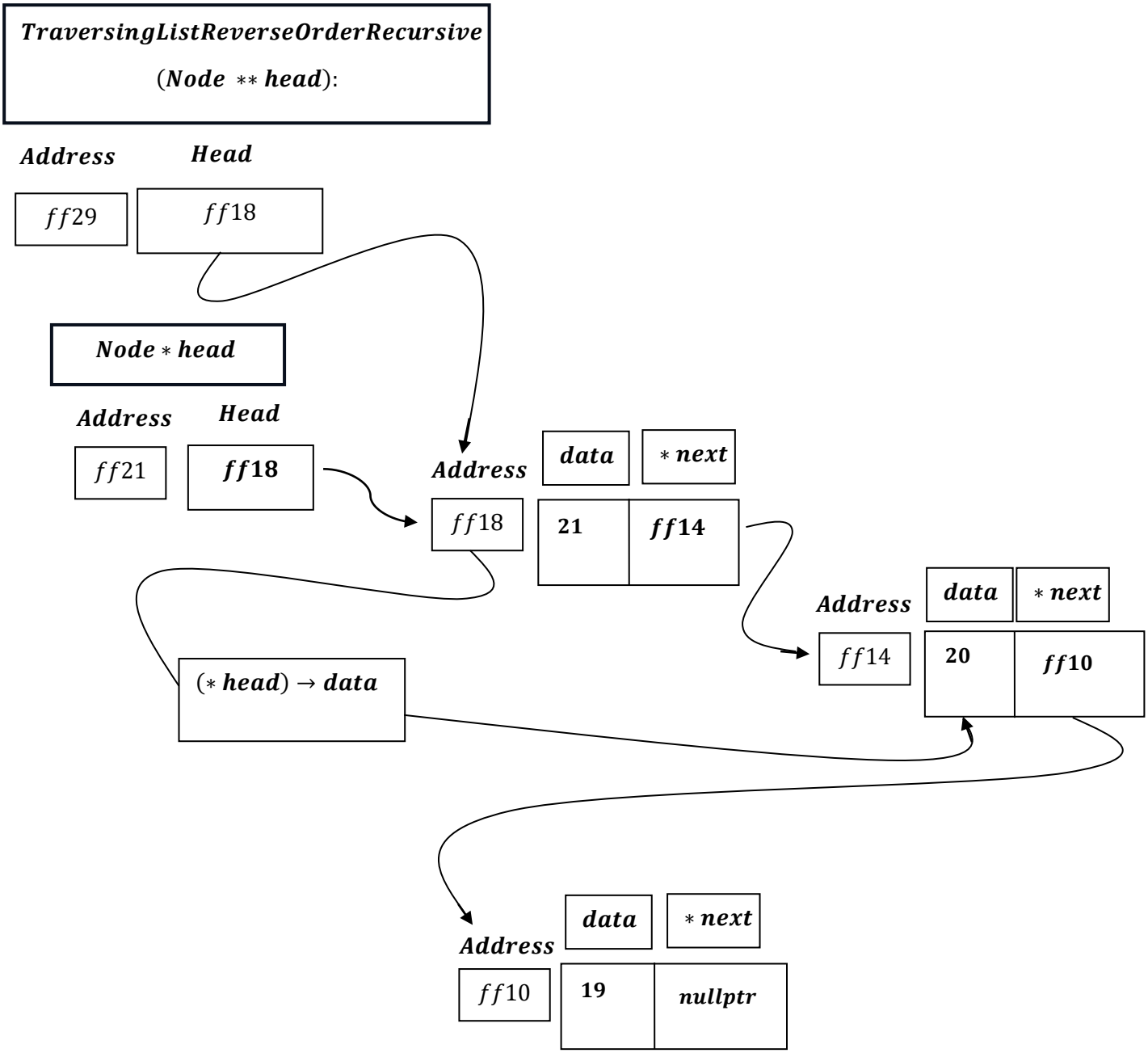


$\boldsymbol{Now, in\ the\ stack\ it\ remains, Node ** head = ff18, hence\ Node ** head\ will\ get\ assigned\ to\ ff18}$.

$\boldsymbol{i.e.}$

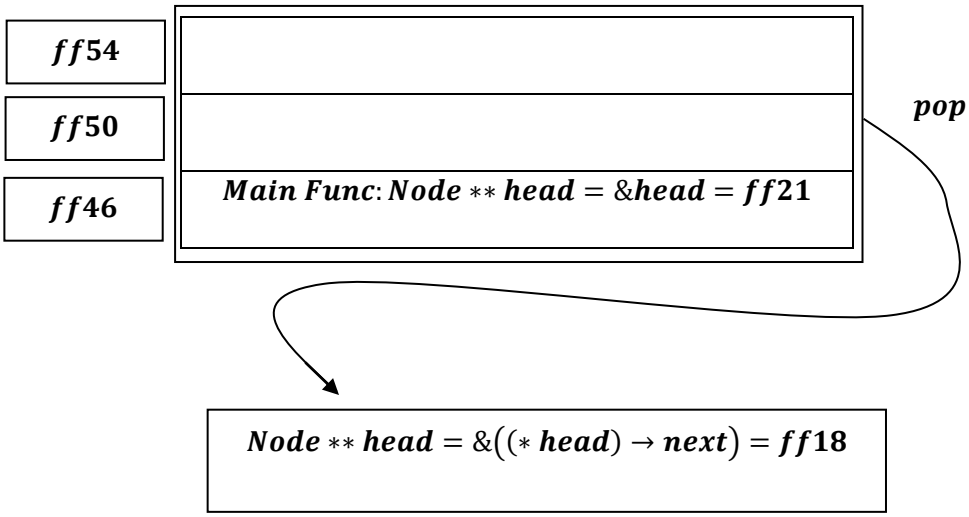

TraversingListReverseOrderRecursive
(Node ** head):
Address
Head
ff29
ff18
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr

$Now, we\ will\ print\ (*\ head) \rightarrow data;$



$Hence\ (*\ head) \rightarrow data;\ will\ print\ 20.$

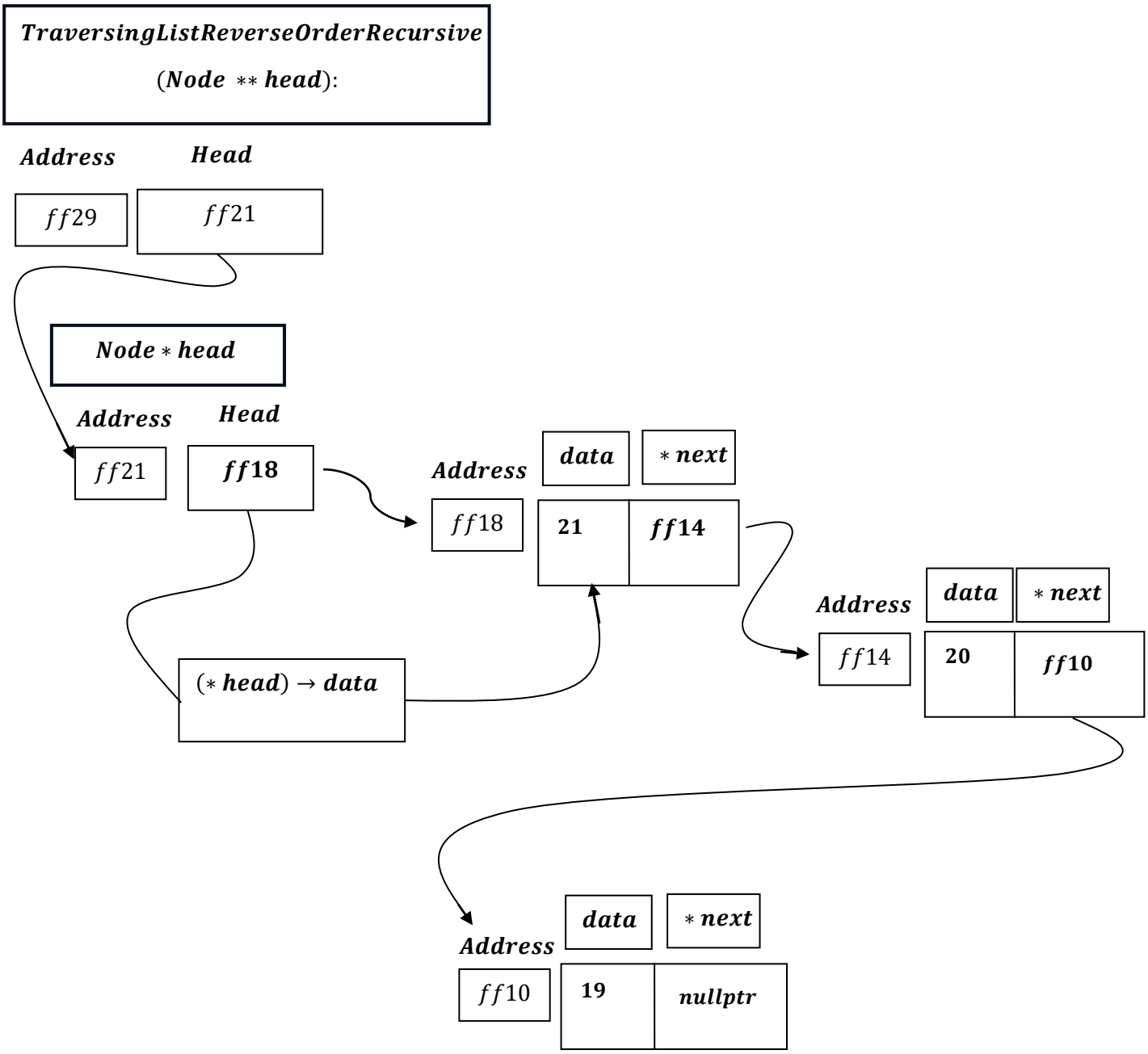
$\boldsymbol{Next, Node ** head = \&((* head) \rightarrow next) = ff14, will\ pop\ out\ from\ the\ stack}$.



$\boldsymbol{Now, in\ the\ stack\ it\ remains, Node ** head = ff21, hence\ Node ** head\ will\ get\ assigned\ to\ ff21}$.

$\boldsymbol{i.e.}$

$Now, we\ will\ print\ (*\ head) \rightarrow data;$



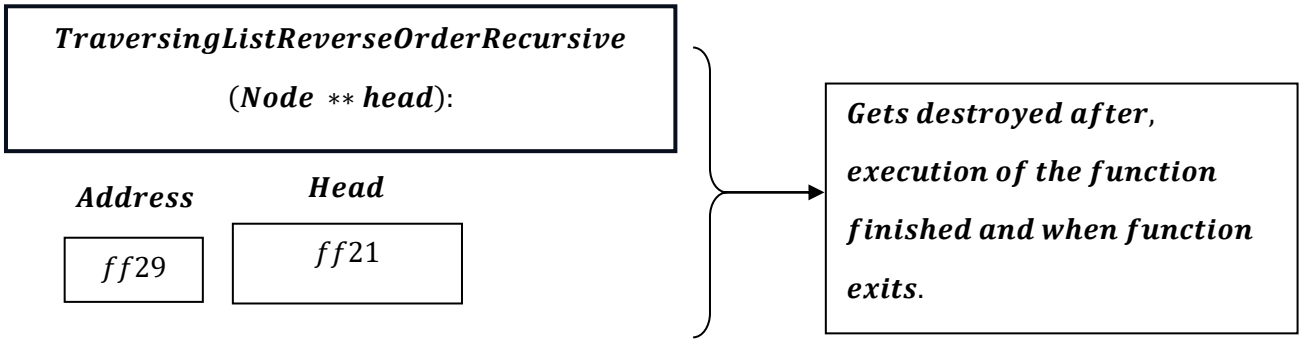
$As\ Node\ **\ head = \&head\ i.e\ address\ of\ Node\ *\ head = ff21$, $now\ hence,$

$(*\ head) \rightarrow data, here, *\ head\ reflects\ Node\ *\ head\ i.e.\ ff18\ and\ (*\ head) \rightarrow data\ is:$

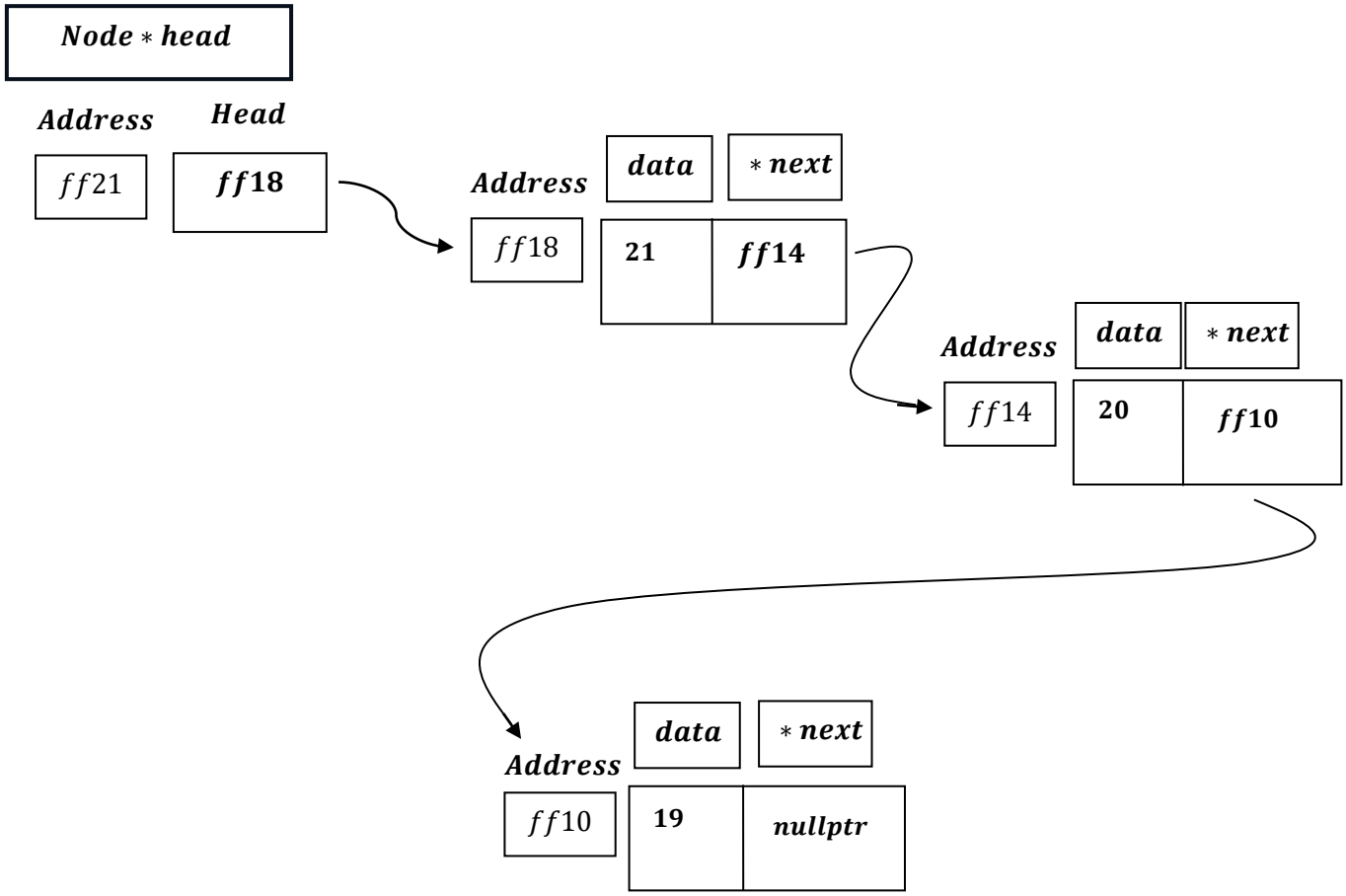
$ff18 \rightarrow data.$

$Hence\ (*\ head) \rightarrow data;\ will\ print\ 21.$

Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



And we are left with:



The function TraversingListReverseOrderRecursive step by step analysis:

1. **Function Signature**: –

- **Parameter**: Node ** head – a pointer to a pointer to the head of the list. This double pointer is used to maintain a reference to the original head node of the list throughout the recursive calls.

2. **Base Case Check for Empty List**: –

- This checks if the list is empty by verifying if the head pointer points to nullptr.
- If the list is empty, it prints "List is empty." and exits the function.

3. **Recursive Case**:

- **Condition**: if ((* head) → next != nullptr):
  - This checks if the current node (* head)is not the last node in the list. If it's not the last node, the function makes a recursive call.

- **Recursive Call**: TraverseListReverseOrderRecursive (&((* head) → next)):
  - The function is called recursively with the address of the next pointer of the current node. This moves the recursion one step further along the list.
  - This recursive call continues until the end of the list is reached (i.e., when (* head) → next is nullptr).

- **Print Statement**: cout << (* head) → data << ;
  - After the recursive call returns (which happens when the end of the list is reached and the recursion unwinds), the data of the current node is printed.
  - This print statement is executed after the recursive call, which means it prints the nodes in reverse order as the recursion unwinds back up the call stack.

$TraversingListReverseOrderRecursive$

$-Time\ Complexity$.

```
void TraverseListReverseOrderRecursive(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {

        if ((*head)->next != nullptr)
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
```

$O(1) i.e. T(0) = 1.$

$T(n-1)$

$Prints\ `n`\ times\ i.e.\ during\ unwinding\ of\ recursion, hence, it\ is\ also\ linked\ with\ recursion.$

$\underline{Best\ Case\ (When\ List\ is\ Empty)}$

$When\ List\ is\ Empty, then, as\ soon\ as\ it\ enters\ the\ function:$

$TraversingListReverseOrderRecursive, if\ checks\ whether\ head\ pointer\ pointed\ to\ ``nullptr``\ or\ not\ , if\ so\ then\ outputs: ``List\ Is\ Empty``\ and\ returns\ nothing\ , and\ then\ exits\ from\ function. This\ takes, constant\ `c`\ amount\ of\ time\ i.e. \Omega(1).$

$Hence\ when\ List\ is\ Empty, we\ get\ our\ best\ case: \Omega(1).$

$\underline{Worst\ Case\ (When\ List\ is\ not\ Empty)}$

$\underline{Base\ Case}$:

$When\ T(0)\ i.e.\ List\ is\ Empty, function\ runs\ once\ which\ we\ can\ say\ , base\ case,$

$T(0) = 1.$

$\underline{Recursion}$:

$The\ recursion\ occurs\ T(n-1)\ times\ i.e.\ if\ n = 3\ then\ 2\ times\ as\ third\ time$

$(*\ head) \rightarrow next;\ will\ be\ ``nullptr``.$

$And, on\ unwinding\ of\ Recursion, each\ time\ it\ prints\ (*\ head) \rightarrow data\ ;.$

$That\ is\ forming : T(n-1) + 1$

$Also, some\ may\ say, using\ Base\ Case: T(n-1) + T(0)\ , but\ more\ precise\ way\ is$

$using, printing\ \ (*\ head) \rightarrow data\ ; on\ unwinding\ of\ recursive\ stack\ .$

$$T(n) = \begin{cases} 1 & , for\ (n = 0) \\ T(n-1) + 1 & , for\ (n > 1) \end{cases}$$

$A)By\ Substitution\ Method$:

$T(n) = T(n-1) + 1$

$Therefore\ , T(n-1) = T(n-1-1) + 1 = T(n-2) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$T(n) = (T(n-2) + 1) + 1$

$Now\ , T(n-2) = T(n-2-1) + 1 = T(n-3) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$T(n) = \big((T(n-3) + 1)\ + 1\big)\ + 1$

$Now\ if\ it\ runs\ upto\ `i`\ times\ we\ get$:

$Now\ , T(n-i) = T(n-i-1) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$T(n) = T(n-i-1) + 1 + 1 + 1 + \cdots i\ times$

$When\ i = n-1\ we\ get$:

$T(n) = T(n-(n-1)-1) + 1 + 1 + 1 + \cdots (n-1) times$

$= T(n-n+1-1) + 1 + 1 + 1 \ldots . (n-1) times$

$= T(0) + 1 + 1 + 1 \ldots .\ \ (n-1) times$

$And\ we\ know\ T(0) = 1\ added\ to\ 1 + 1 + 1 + \cdots (n-1) times\ \ it\ becomes$:

$= 1 + 1 + 1 + \cdots n\ times$

$i.e. 1 \times n = n\ and\ time\ complexity\ is$:

$= O(n)$.

$B)$ Recursion Tree Method:

$$T(n) = \begin{cases} 1 & , for\ (n = 0) \\ T(n-1) + 1 & , for\ (n > 0) \end{cases}$$

$We\ know$:

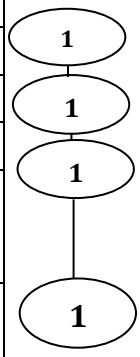
$$t_n = \begin{cases} 1 & , \quad for\ n = 1 \\ t_{n-1} + a & , \quad for\ n > 1 \end{cases}$$

$When\ a\ is\ 1\ , problem\ size\ becomes\ 1\ i.e. constant.$

$Therefore, the\ recurrence\ tree\ for\ this\ recurrence\ equation$

$would\ be\ shown\ below$:

$Level$	$No.of\ problems$	$Problem\ Size$	$Work\ done = Problem\ Size \times No.of\ Problems$
0	1	1	$1 \times 1 = 1$
1	1	1	$1 \times 1 = 1$
2	1	1	$1 \times 1 = 1$
. .	. .	. .	. .
$n-1$	1	1	$1 \times 1 = 1$



$One\ can\ observe\ that\ the\ work\ done\ at\ every\ level\ is\ 1\ and\ the\ size\ of\ the\ problem$
$is\ reduced\ by\ a\ factor\ of\ 1\ at\ every\ level$.

$Therefore, at\ the\ level\ n-1, the\ problem\ size\ and\ work\ done\ would\ be\ 1.$
$The\ total\ number\ of\ levels\ is\ n(as\ the\ level\ of\ the\ root\ is\ 0).$

$Therefore, the\ final\ cost\ can\ be\ estimated\ as\ follows:$

$$Total\ Cost = \sum_{i=1}^{n} 1 = 1 + 1 + 1 + \cdots n\ times = 1 \times n = n.$$

$Therefore, the\ asymptotic\ complexity\ would\ be\ O(n).$

$\underline{Average\ Case(Average\ Case = Worst\ Case)}$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list$
$of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the$
$time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq n \leq c_2 \times n$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$TraversingListReverseOrderRecursive$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case = Worst\ Case$	$\Theta(n)$

****